



Autonomous collaborative rescue of drone swarms in cluttered environments

Xiangyu Zhao¹ · Zipeng Zhao¹ · Yu Wan¹ · Jun Tang¹

Received: 10 November 2024 / Revised: 8 March 2025 / Accepted: 3 May 2025 / Published online: 11 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

With the rapid development of drone technology, the autonomous collaborative control of drone swarms has been extensively applied across various domains. However, in some cluttered environments, drone swarms need to perform rescue missions without centralized control, relying on onboard controllers instead, which brings challenges for the collaboration and obstacle avoidance of drone swarms. In this paper, we propose a flocking control model for drone swarms, Convolutional Soft Actor-Critic (CSAC), based on deep reinforcement learning (DRL). Specifically, we integrate DRL with convolutional neural networks (CNNs), aiming to accelerate the end-to-end decision-making process by reducing model parameters. With this framework, each drone can independently make decisions based on its current state and environmental information, ensuring the autonomous execution of collaborative rescue missions in cluttered environments. Through simulation experiments, we have validated the effectiveness of the model. Besides, compared to the baseline algorithm, our model improves decision performance by 5.90% while reducing the number of parameters by 92.91%.

Keywords Drone swarms · Rescue mission · Autonomous collaboration · DRL · Obstacle avoidance · CNNs

1 Introduction

With the rapid development of technology, drones have become a focal point in the fields of robotics and artificial intelligence. They are capable of performing search and rescue missions on land and at sea [1], as well as reconnaissance tasks [2–5], and engaging in high-risk jobs [6–11], showcasing substantial socio-economic potential. When faced with complex tasks, the collaborative capabilities of drone swarms stand out. Compared to a single drone, a drone swarm not only reduces the time required to

complete tasks but also enhances the system's robustness and expands the operational range [12]. However, the autonomous collaboration of a drone swarm during mission execution has always been a hot topic among researchers [13, 14]. They must avoid collisions with buildings and other drones while trying to complete the mission as quickly as possible, which undoubtedly adds to the difficulty and challenge of the mission.

For single and multiple agents, many works have proposed effective methods for their obstacle avoidance. The current research mainly focuses on the identification of obstacles and collision avoidance for single drones. To enhance obstacle avoidance efficiency and execution speed, [15–17] utilized deep learning models to map drone states directly to control actions, while [18, 19] integrated deep learning for object detection, image processing, and RGB-D information fusion, along with memory-enhanced learning. Based on DRL, [20] combined Faster-RCNN and data storage mechanism to improve deep Q-network (DQN), improved the training speed and released the memory of experience storage. Similarly, [21] proposed an algorithm based on probability and DRL, which

✉ Yu Wan
wanyu13@nudt.edu.cn

Xiangyu Zhao
zxy0822@nudt.edu.cn

Zipeng Zhao
zhaozipeng22@nudt.edu.cn

Jun Tang
tangjun06@nudt.edu.cn

¹ College of Systems Engineering, National University of Defense Technology, Changsha 410073, China

accelerated the convergence speed of neural networks and improved the generalization ability of the agent. Autonomous obstacle avoidance in complex environments with multiple moving obstacles was implemented through DRL-based subentry points and grid-based path planning [22, 23]. [24] proposed a reactive online obstacle avoidance decision-making mechanism based on DRL. In [25], a learning method that combines navigation and collision avoidance schemes based on reinforcement learning with low-level and high-level control is proposed to bridge the gap between simulation and reality. A single drone has made significant strides in various fields, demonstrating adeptness in performing missions with precision and efficiency. However, as the missions become more complex, the limitations of a single agent become apparent. Drone swarms offer a solution by enhancing system resilience, expanding operational capabilities, and reducing mission completion times.

Flocking control plays a key role in drone swarm control field, among which decentralization or elimination of system collapse caused by single point of failure has always been a research focus. It should not only avoid collision with obstacles, but also maintain the stability of the internal structure of the swarm. Drone swarm control technologies had been advancing with a focus on swarm intelligence to prevent system collapse due to single points of failure and to maintain the stability of the swarm's internal structure [26, 27]. Some advanced method like [28, 29] introduced optimization theory into DRL for real-time collision avoidance. In [30], the fully connected neural networks (FCNNs) in the DRL model was changed into a long and short term memory network to improve the scalability of the number of drones in the swarm. Real-time localization and swarm maintenance had been achieved through the use of YOLOv3 and Reynolds criterion-based flocking algorithms [31]. [32–34] used the centralized training with decentralized execution (CTDE) paradigm to train the drone swarm, improve training efficiency and maintain the stability of the internal structure of the swarm. [35] applied the curriculum learning paradigm to the DRL algorithm, so that the agent can learn the flocking obstacle avoidance policy efficiently and stably. The flocking control of drone swarms has become increasingly mature. Towards rescue missions, many methods exhibit good performance, but they have a high number of model parameters, leading to substantial computational requirements. In addition, the maintenance cost for drones is also uncontrollable. Since the effect of an algorithm is unknown, if there is a collision in the physical world, there is a high probability that the drone will be damaged.

To solve the above challenges, we propose CSAC based on DRL algorithm Soft Actor-Critic (SAC) [32]. Besides, we have built a multi-obstacle simulation environment to

simulate a rescue mission in cluttered environments. SAC is a widely used off-policy deep reinforcement learning algorithm. It balances exploration and exploitation by maximizing a complex objective function that combines expected returns and entropy. However, the algorithm has a large number of parameters when dealing with high-dimensional continuous action spaces, leading to high computational costs. To address this, we introduced CNNs.

For the structure of actor and critic of SAC, in order to reduce the parameters of the model and speed up the end-to-end decision-making time, we use CNNs instead of FCNNs. Furthermore, we have redefined and designed the observation space, action space and reward function to ensure stable control of drone swarms in the simulation environment. After that, we validate the performance of our model in the simulation environment and then compared with the benchmark algorithms SAC and proximal policy optimization (PPO) [36]. The results indicating that the model can effectively be applied to drone swarms to ensure the completion of collaboration, obstacle avoidance, and rescue missions in cluttered environments. The main contributions can be summarized as:

- We propose CSAC for drone swarms based on DRL and CNNs to address the challenge of autonomous collaborative operations of drone swarms for rescue missions in cluttered environments.
- We build a multi-obstacle simulation environment to simulate a rescue mission in cluttered environments. By refining and designing the observation space, action space and reward function, we make sure the stable control of drone swarms.
- We apply CSAC, SAC, and PPO to our simulation environment. Compared with SAC, our method reduces the mission execution time by 5.90% while reducing the number of parameters by 92.91%.

This paper is organized as follows. Section 2 introduces problem scenario, drone dynamics model, drone perception model. In section 3, DRL and the proposed model is explained. The detailed experimental results and analysis are displayed in Sect. 4. Finally, conclusions and future research are summarized in Sect. 5.

2 Background

2.1 Problem scenario

We design a multi-obstacle scenario containing multiple drones, denoted as $\mathcal{D} (u_i \in \mathcal{D}, i = 1, 2, 3, \dots, N)$, where N is the number of drones. In the following, we will refer to these drones collectively as a drone swarm. In the scenario, we randomly initialize the position of the drone swarm and

the rescue area, and the drone swarm needs to complete the mission of crossing multiple obstacles from the initial position to the rescue area. In addition, the scenario is a communication-denied zone, meaning that drones can only communicate with each other through the actions they perform. Within the drone swarm, drones can only communicate and collaborate with other drones within their visual range, which is the range of radar detection or onboard cameras. To reduce the risk of collisions, the drone swarm should complete its mission as quickly as possible while ensuring safety, which includes obstacle avoidance and collision avoidance. The specific scenario is defined as shown in Fig. 3.

2.2 Drone dynamics model

In the scenario, we assume that the drone swarm performs its mission at a fixed height, so we only need to describe the dynamics model of the drone in the 2-dimensional scenario:

$$\begin{cases} \dot{x}_i = v_{x_i}, \\ \dot{y}_i = v_{y_i}, \\ \dot{v}_{x_i} = (v_{x_i}^t - v_{x_i}^{t-1})/\Delta t, \\ \dot{v}_{y_i} = (v_{y_i}^t - v_{y_i}^{t-1})/\Delta t, \end{cases} \quad (1)$$

where x_i and y_i are the positions of the drone i in the x -axis and y -axis directions, v_{x_i} and v_{y_i} are the velocity of the drone i in the x -axis and y -axis directions, and $v_{x_i}^t$ and $v_{x_i}^{t-1}$ are the velocity of the drone i at time t and $t - 1$ in the x -axis direction, Δt is a fixed time constant.

2.3 Drone perception model

The visual perception of the drone is limited by angle range ϕ_v and distance range d_{max} . Other drones or obstacles within this range will be considered in distance estimation and collision judgment. In this way, information redundancy can be avoided and system stability can be ensured [37, 38]. In addition, the visual perception direction of the drone is consistent with its velocity direction by default. Therefore, in the whole process of executing the mission, it will automatically adjust the direction of visual perception according to the direction of velocity, so that there is no need to consider the direction of visual perception [32]. The velocity direction ϕ_i of drone i is determined by the velocity components in the x and y directions:

$$\phi_i = \arccos \frac{\mathbf{v}_{x_i} \cdot \mathbf{v}_{y_i}}{|\mathbf{v}_{x_i}| \cdot |\mathbf{v}_{y_i}|} \quad (2)$$

where $|\cdot|$ is the magnitude of the corresponding vector. The drone model is shown in Fig. 1.

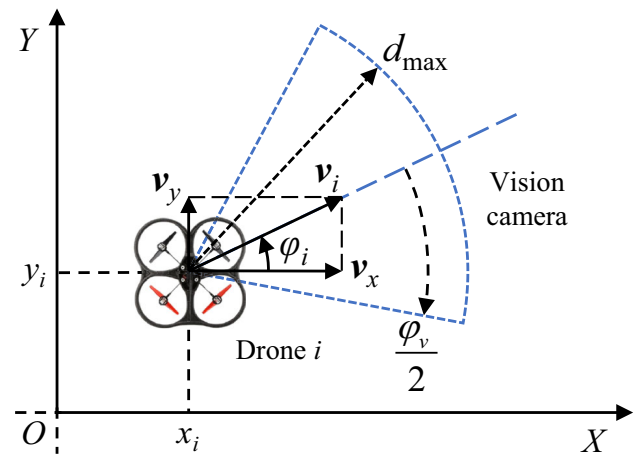


Fig. 1 The dynamics model of drone

3 Method

3.1 Deep reinforcement learning

DRL uses Partially Observable Markov Decision Process (POMDP) as a framework used to solve decision problems with uncertainty for multi-agents. This method is mainly applicable to the fields of multi-agent cooperative control and unmanned driving [39].

We describe the POMDP as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \mathcal{O} \rangle$, where $\mathcal{S}$ is a state set, $\mathcal{A}$ is a action set, $\mathcal{P}$ is the transition probabilities between states $P(\mathcal{O}_{t+1} = o' | \mathcal{O}_t = o, \mathcal{A}_t = a)$, $\mathcal{R}$ is the reward function $R(\mathcal{O}_t = o, \mathcal{A}_t = a)$, γ is the discount factor, and $\mathcal{O}$ is an observation set. For brevity, in the following we will omit the drone index dependency when clear from the context.

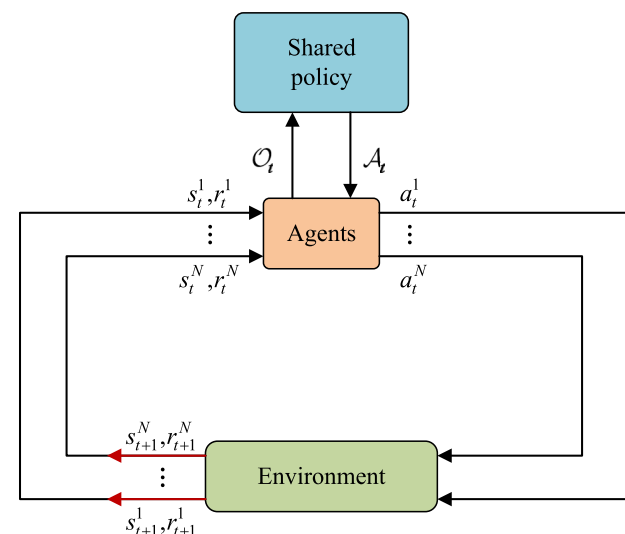


Fig. 2 The frame work of DRL

As shown in Fig. 2, at each time step t , a drone observes the environmental state s_t , and generates local information $o_t \sim O(s_t)$, where $O(\cdot)$ is an observation function. Then the drone takes action $a_t \in \mathcal{A}$ according to its policy $a_t \sim \pi(a_t | o_t)$. After taking the action, the environment state is changed to $o_{t+1} \in \mathcal{O}$ based on the transition probabilities between states $P(o'_{t+1} | o_t, a_t)$, and the agent receives a reward $r_t \in R(o_t, a_t)$. The policy π is the probability distribution of all actions in a given state, i.e. $\pi(a_t | o_t) = P(\mathcal{A}_t = a | \mathcal{O}_t = o)$. The agent is mainly to learn the optimal policy π^* to complete the assigned mission. In order to evaluate the quality of the policy, the state value function $V_\pi(o)$ and the state-action value function $Q_\pi(o, a)$ are introduced,

$$\begin{aligned} V_\pi(o) &= \mathbb{E}[G_t | \mathcal{O}_t = o], \\ Q_\pi(o, a) &= \mathbb{E}[G_t | \mathcal{O}_t = o, \mathcal{A}_t = a] \end{aligned} \quad (3)$$

where G_t is the return at the time step t , represents the long-term cumulative reward from a certain state to the final state,

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = \sum_{i=0}^{\infty} \gamma^i R_{t+i}, \quad (4)$$

$V_\pi(o)$ is the expected value of G_t if the current state o follows the policy π , and $Q_\pi(o, a)$ is the expected value of G_t when taking the action a in state o .

Combining the Bellman equation with (3), $Q_\pi(o, a)$ can be represented as follows,

$$\begin{aligned} Q_\pi(o, a) &= R(o, a) + \\ &\gamma \sum_{o' \in \mathcal{O}} P(o' | o, a) \sum_{a' \in \mathcal{A}} \pi(a' | o') Q_\pi(o', a') \end{aligned} \quad (5)$$

After using (4) to estimate $Q_\pi(o, a)$, we can get the optimal policy by maximizing the state-action value function,

$$\pi^*(a | o) = \begin{cases} 1, & \text{if } a = \arg \max_{a \in \mathcal{A}} Q_*(o, a) \\ 0, & \text{else} \end{cases} \quad (6)$$

where $Q_*(o, a) = \max Q_\pi(o, a)$ is the maximum value of the state-action value function. In the autonomous collaboration of the drone swarm, once the policy is determined, each agent can take the optimal action according to the current partially observable state to ensure the safe completion of the mission.

3.2 Convolutional soft actor-critic (CSAC)

To inherit the superior characteristics of the SAC algorithm in balancing exploration and exploitation, we retained its core framework, ensuring that the model maximizes expected returns while maintaining necessary exploratory behavior. CNNs are capable of automatically capturing the

spatial hierarchy of input data through convolutional layers, extracting local features using convolution kernels, and reducing the spatial dimensions of features through pooling layers, thereby enhancing the model's invariance to translation. Compared to fully connected networks, CNNs significantly reduce the number of connections between each neuron and the input data through local connections, thus drastically reducing the number of model parameters. Based on this, we improved the structure of the Actor and Critic networks in SAC, proposing CSAC. This model optimizes the overall structure, as illustrated in Fig. 3. The detailed comparison of network model structures in SAC and CSAC is shown in Table 1.

3.2.1 Observation space

At the time step t , the observation space of the current drone, $\mathbf{o}_t = [\mathbf{o}_t^g, \mathbf{o}_t^v, \mathbf{o}_t^a, \mathbf{o}_t^{\text{neig}}, \mathbf{o}_t^{\text{obs}}]$, consists of the relative position of the target $\mathbf{o}_t^g$, the velocity $\mathbf{o}_t^v$ and the acceleration $\mathbf{o}_t^a$ of the drone, the states of other drones $\mathbf{o}_t^{\text{neig}}$ and obstacles $\mathbf{o}_t^{\text{obs}}$ within the visual field. The specific observation of the current drone is defined as shown in Fig. 4.

The observation $\mathbf{o}^g = [d_x, d_y]$ is the relative distance between the current drone and the center of the rescue area, which can be calculated as follows:

$$\begin{cases} d_x = x_{\text{target}} - x, \\ d_y = y_{\text{target}} - y, \end{cases} \quad (7)$$

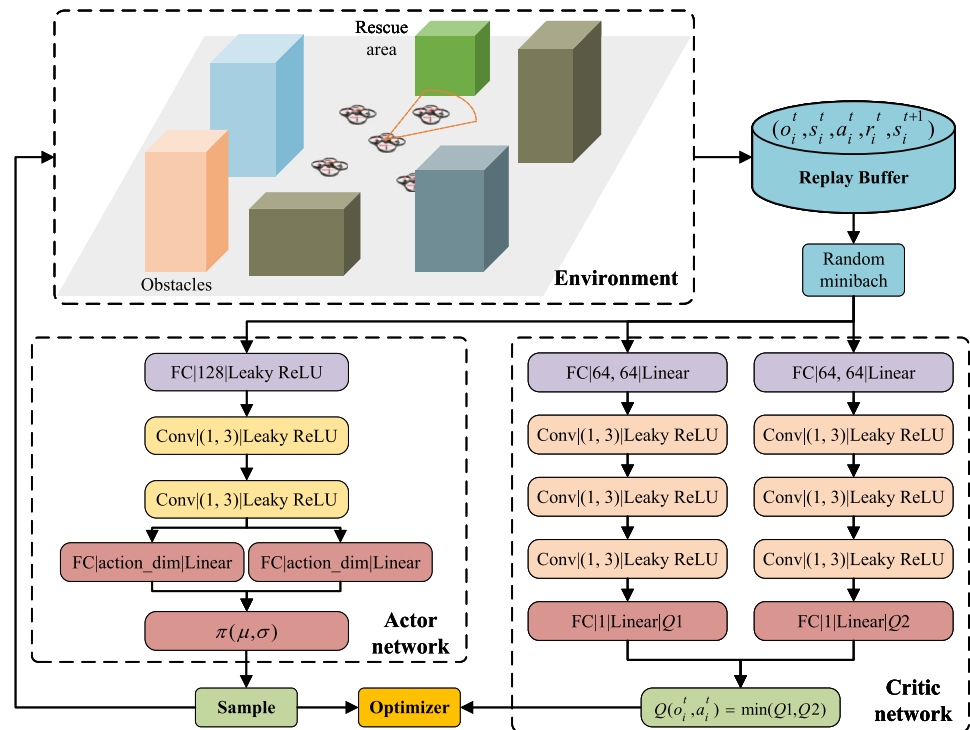
where (x, y) and $(x_{\text{target}}, y_{\text{target}})$ are the coordinates of the drone and the center of the rescue area, respectively.

The observation $\mathbf{o}^v = [v_x, v_y]$ and $\mathbf{o}^a = [a_x, a_y]$ are the speed and acceleration speed of the current drone in the x -axis and y -axis direction, respectively. The observation $\mathbf{o}^{\text{neig}} = [d_1^{\text{neig}}, d_2^{\text{neig}}, \dots, d_n^{\text{neig}}]$ is the distance between the current drone and n drones that meet the following drone's conditions:

- The angle between the direction of the current drone pointing to the drone and the current drone's velocity is less than $\frac{\varphi_v}{2}$.
- The distance between the drone and the current drone is less than d_{max} .
- There is no obstacle between the drone and the current drone.

The observation $\mathbf{o}^{\text{obs}} = [d_1^{\text{obs}}, d_2^{\text{obs}}, \dots, d_n^{\text{obs}}]$ is the distance between the current drone and n obstacles that meet the following obstacle's conditions:

- The angle between the direction of the current drone pointing to the obstacle and the current drone's velocity is less than $\frac{\varphi_o}{2}$.

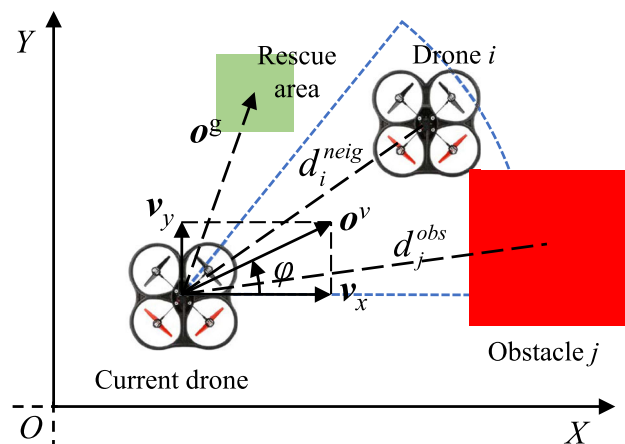
Fig. 3 CSAC model architecture**Table 1** Comparison of algorithm complexity between SAC and CSAC

Network	Output size	SAC	CSAC (our model)
Actor	1×128	128-d fc	128-d fc
	1×128	128-d fc	1×3 conv
	1×64	64-d fc	1×3 conv, avgpooling
	1×4	4-d fc*	4-d fc*
Critic	1×128	128-d fc	128-d fc
	1×128	128-d fc	1×3 conv
	1×64	64-d fc	1×3 conv
	1×32	32-d fc	1×3 conv, avgpooling
	1×1	1-d fc*	1-d fc*
Params		5.5×10^4	3.9×10^3
FLOPs		1.1×10^5	1.1×10^4

The value in bold indicates better performance

* denotes that the corresponding layer does not employ an activation function, while all other layers utilize the LeakyReLU activation function

- The distance between the obstacle and the current drone is less than $d_{\max}$.
- To reduce unnecessary calculations, we only consider the two closest obstacles to the current drone.

**Fig. 4** The observation model of drone

3.2.2 Action space

At the time step t , the action space of the current drone, $\mathbf{a} = [a_x, a_y]$ ($a_x, a_y \in [-a_{\max}, a_{\max}]$), is the acceleration of the drone in the x -axis and y -axis direction, which can ensure the coherence of the drone's movements and makes it closer to the physical world. The value $a_{\max}$ is the maximum acceleration speed of the drone in the environment. Thus, the speed of the drone is updated as follows:

$$\begin{cases} v_x^t = v_x^{t-1} + a_x \cdot \Delta t, \\ v_y^t = v_y^{t-1} + a_y \cdot \Delta t. \end{cases} \quad (8)$$

Besides, the speed should satisfy $v_x, v_y \in [-v_{\max}, v_{\max}]$, where $v_{\max}$ is the maximum speed of the drone in the environment.

3.2.3 Network architecture

SAC is a prevalent off-policy DRL algorithm known for its robust performance in various applications. However, SAC traditionally employs FCNNs for both its actor and critic components. The training of these networks involves fitting a large number of parameters, which can lead to extended end-to-end decision-making times, potentially compromising optimality in terms of speed.

To address this issue, we integrate CNNs with FCNNs to reconstruct the actor and critic, as shown in Fig. 3. In the actor network, the drone's observation o_t passes through four neural networks layers: a linear layer Linear, two convolutional layers and another linear layer. Each layer has 128, 128, 64, 4 nodes, respectively. The first three layers utilize LeakyReLU activation function and the final layer opt instead for a direct mapping through FCNNs. The actor network outputs the mean and standard deviation of the action a_t of the drone, for action sampling, to ensure the model retains exploratory capabilities.

The critic network is designed similar to the actor network. Additionally, to ensure the comprehensive extraction of features, the convolutional layers in our model all employ one-dimensional convolutions with a kernel size of 1×3 , and the number of channels is consistently set to 1. The specific structure of each layer and the dimensions of the output features are as shown in Table 1.

Comparing the algorithm complexity metrics of the algorithms SAC and CSAC, Params and FLOPs (Table 1), it is clearly visible that all the metrics of CSAC are one order of magnitude less than those of SAC. Specifically, the number of parameters in the model is reduced by 92.91% compared to SAC. This indicates that our approach not only significantly decreased the model's parameters but also enhanced its computational efficiency, thus preliminarily fulfilling the objectives of our research.

3.2.4 Reward function

Reward function is the most important component of DRL, and its setting directly determines the difference in actions taken by the agent in the process of interacting with the environment. The adjustment of policy learning is mainly to adjust the relative value of the reward function, increase or decrease the constraint terms. The purpose of setting the reward function in this paper is to enable the drone swarm to learn the flocking control policy for autonomous rescue. Therefore, we designed the reward function as follows:

$$r_t = r_t^g + r_t^v + r_t^u + r_t^b \quad (9)$$

where r_t is the reward received by the drone at the time step t , which includes four terms, namely, the reward related to rescue area r_t^g , the reward related to drone i 's velocity r_t^v , the reward related to the collision avoidance within the drone swarm r_t^u , and the collision avoidance reward r_t^b , is the reward related to the collision avoidance of obstacles.

The target-reaching reward r_t^g motivates the drone i to approach or reach the rescue area, and is designed as follows:

$$r_t^g = \begin{cases} 100, & \text{if } d_t^g \leq d_{\max}^g \\ 2 * \text{clip}(v_t^g, -\infty, 2), & \text{elif } \varphi_t^g \leq 40 \\ 2 * \text{clip}(v_t^g, -\infty, 2) - 1, & \text{elif } \varphi_t^g \leq 70 \\ 2 * \text{clip}(v_t^g, -\infty, 2) - 2, & \text{else} \end{cases} \quad (10)$$

where d_t^g is the distance between the current drone and the center of the rescue area, v_t^g is the target speed of the current drone, i.e. $v_t^g = |v| \cdot \cos \varphi_t^g$, φ_t^g is the angle between the current drone's velocity v and the direction of the current drone pointing to the target, $\text{clip}(\cdot)$ is a clipping operation as follows:

$$\text{clip}(v_t^g, -\infty, 2) = \begin{cases} v_t^g, & \text{if } v_t^g \leq 2 \\ 2, & \text{else} \end{cases} \quad (11)$$

Before reaching the rescue area, the velocity reward r_t^v encourages the drone to fly at maximum speed, but not beyond it, ensuring that its flight meets the practical requirements of the dynamics. After reaching the rescue area, it punishes the drone to accelerate until it hovers within the rescue area, reducing the risk of collision. It is designed as follows:

$$r_t^v = \begin{cases} 10, & \text{if } d_t^g \leq d_{\max}^g, |v| \leq 0.1 \\ -20, & \text{if } d_t^g \leq d_{\max}^g, |v| > 0.1 \\ |v|, & \text{if } d_t^g > d_{\max}^g, |v| \leq 2.2 \\ |v| - 1, & \text{if } d_t^g > d_{\max}^g, |v| > 2.2 \\ |v| - 2, & \text{if } d_t^g > d_{\max}^g, |v| > 2.5 \end{cases} \quad (12)$$

The flocking control reward r_t^u ensures the stability of the internal structure of the drone swarm, neither colliding nor being too far apart, and is designed as follows:

$$r_t^u = \begin{cases} -50, & \text{if } d_t^u \leq 0.5 \\ -8 * \text{count}, & \text{elif } d_t^u \leq 1.3 \\ -4 * \text{count}, & \text{elif } d_t^u \leq 1.7 \\ 0, & \text{elif } 1.7 < d_t^u \leq 4.5 \\ -2, & \text{else} \end{cases} \quad (13)$$

where d_t^u is a set of the distance between the current drone and the drone that meet the drone's conditions at the time

step t , $count$ is the number of drones that meet the corresponding distance condition.

The collision avoidance reward r_t^b penalizes the drone for colliding with obstacles, and keeps a safe distance from the obstacle to eliminate the risk of collision. It is designed as follows:

$$r_t^b = \begin{cases} -100, & \text{if } d_t^b \leq 1.2 \\ -30, & \text{elif } d_t^b \leq 1.5 \\ -20, & \text{elif } d_t^b \leq 2.2, v_t^s > 0.01 \\ 0, & \text{else} \end{cases} \quad (14)$$

where d_t^b is a set of the distance between the current drone and the obstacles that meet the obstacle's conditions at the time step t .

4 Experiments and results

4.1 Simulation environment and training setup

In order to achieve robust learning of flocking control policies, we use ROS melodic to implement our algorithm and build a scenario on rviz for algorithm training. To simplify the problem, we treat each drone as a mass point, each obstacle as a cylinder, and the rescue area as a circular area with a fixed height centered at a certain point. We use pytorch to build flocking algorithms for the drone swarm based on CSAC, SAC, and PPO, respectively, and train them on a laptop with Intel i9-13980HX and NVIDIA GeForce RTX 4080.

During the training process, the initial position of the drone swarm and the rescue area of the mission are paired, that is, if the rescue area is determined, then the initial position of the drone is also determined. Specifically, we define this correspondence as the drone swarm flying from the midpoint of one side of the square area to the midpoint of the opposite side (rescue area). It is worth noting that in order to ensure that the drone can fully explore its observation space, in each episode, we randomly select the rescue area from the candidate positions to improve the robustness of the model [40]. The parameters for policy training are listed in Table 2.

From Fig. 5, it is evident that CSAC converges after approximately 1000 episodes. On the other hand, SAC takes around 3000 episodes to start converging, and its converged value is slightly lower than CSAC, with an oscillatory and unstable convergence trend. In comparison, PPO has negative rewards for the first 6000 episodes, indicating that it cannot safely completed the search and rescue mission. During actual training, PPO only begins to converge after about 9000 episodes, with a converged value close to SAC. Moreover, it is observed that only

Table 2 Parameters of training

Parameters	Value	Parameters	Value
Max Episodes	6000	Batch Size	300
Max Steps	500	$a_{\max}$	0.8m/s^2
Discount Factor γ	0.99	N	5
Soft Update Factor τ	0.005	Δt	0.5s
Actor Learning Rate	1×10^{-4}	$d_{\max}^g$	2.0m
Critic Learning Rate	1×10^{-4}	ϕ_v	0.5π
α Learning Rate	3×10^{-4}	$d_{\max}$	7 m
Reply Buffer Size	5×10^5	$v_{\max}$	2 m/s

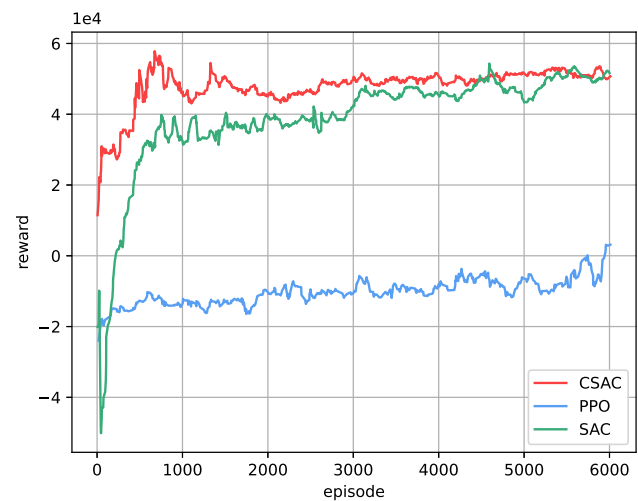


Fig. 5 Rewards for each episode during the training process

CSAC has a positive reward at the beginning of training. CNNs inherently prioritize low-level spatial relationships over global correlations and avoid overfitting to noisy initial exploration trajectories. This also indicates that our algorithm can adapt more effectively to new environments and has superior abilities in exploring and utilizing data. Subsequent tests of each flocking algorithm are conducted using the convergent models.

4.2 Performance metrics and experiment scenarios

In order to compare the performance of different algorithms in our experimental scenarios, we use the following quantitative evaluation metrics:

- Average speed v : The average speed of all drones, which is used to evaluate the path planning effectiveness of the drone swarm.

- $d_{\min}^u$: The minimum distance between all drones which is used to evaluate the cooperative performance of the drone swarm.
- $d_{\min}^{\text{obs}}$: The minimum distance between drones and obstacles, which is used to evaluate the obstacle avoidance ability of the drone swarm.
- Success Rate: The ratio of drones that reach the rescue area without any collision within a limited time.
- T^g : The mission time, which is used to evaluate the mission execution efficiency of the drone swarm.

In our experiment, we construct four test environments: there are 9 cylindrical obstacles in each scenario, and the rescue areas are in the north direction (S1), south direction (S2), west direction (S3) and east direction (S4) of the scenario, respectively.

4.3 Testing and results

In each scenario, we randomly test the flocking algorithms with the same random seed as reported in 4.1. The results are listed in Table 3.

Across the four scenarios, all three algorithms demonstrate excellent success rates in accomplishing the rescue missions. However, PPO shows variable success rates, indicating that the flocking policies it generates are not highly stable and do not adapt well to different environments. In contrast, both SAC and CSAC maintain success rates above 95%, with CSAC achieving and sustaining a success rate of over 98%. Furthermore, for the metric T^g , we can draw the same conclusions as before. It can be seen that our algorithm is capable of adapting to different scenario changes and effectively stabilizing the reduction of mission time.

In S1, compared to SAC, PPO completes the mission more rapidly, but during the coordination process, to avoid collisions, one drone arrives at the rescue area from the opposite direction, as shown in Fig. 6a, which results in suboptimal mission completion time. In S2, SAC outperforms PPO overall, thanks to its enhanced exploration capabilities and robustness achieved by incorporating a temperature parameter. In both scenarios, CSAC outperforms the other two algorithms across all metrics, reducing the mission completion time by approximately 12.24% and 2.33% compared to SAC. This indicates that under CSAC, the drone swarm demonstrates stable performance and is capable of autonomously accomplishing coordinated rescue missions.

In S3, by examining Table 3, we observe that under PPO, the drone swarm maintains the smallest distance from obstacles, yet its mission completion time T^g is the longest. Looking at Fig. 6c, we identify collisions between drones and obstacles, suggesting that PPO does not sufficiently consider the safety of the drone swarm, leading to flawed decision-making and consequently prolonging the mission completion time. In comparison, the other two algorithms effectively balance mission completion with safety. Notably, the drone swarm under CSAC not only reduces the mission completion time by approximately 6.67% but also exhibits smoother flight trajectories, more closely resembling real-world flight conditions.

In S4, PPO's performance is close to that of CSAC, even surpassing CSAC in terms of speed and compactness, but it falls short due to lower and less success rates. Both PPO and CSAC complete the mission in the same amount of time, which is 10.5 s, representing an improvement of approximately 2.38% over SAC. Observing Fig. 6d, h, and i, it is evident that both drone swarms excellently

Table 3 Comparison results of three policies under four different scenarios

Scenario	Policy	$v(\text{m/s})$	$d_{\min}^u(\text{m})$	$d_{\min}^{\text{obs}}(\text{m})$	$T^g(\text{s})$	Success Rate
S1	PPO	1.043±0.617	2.103±0.718	3.753±1.194	11.50	88.7%
	SAC	0.976±0.538	1.557±0.615	3.550±0.836	12.25	98.5%
	CSAC	1.079±0.603	1.501±0.746	3.426±0.766	10.75	99.1 %
S2	PPO	0.987±0.667	2.079±0.986	3.532±0.925	11.50	85.6%
	SAC	1.056±0.588	1.718±0.794	3.487±0.804	10.75	97.9%
	CSAC	1.124±0.583	1.631±0.689	3.474±0.756	10.50	98.5 %
S3	PPO	0.509±0.405	1.568±0.697	3.078±1.115	24.75	81.3%
	SAC	1.075±0.594	1.526±0.671	3.580±0.800	11.25	98.1%
	CSAC	1.120±0.618	1.516±0.770	3.347±0.760	10.50	98.7 %
S4	PPO	1.132±0.652	1.421±0.565	3.638±0.912	10.25	91.9%
	SAC	1.095±0.542	1.949±0.744	3.543±0.814	10.50	95.9%
	CSAC	1.129±0.593	1.629±0.690	3.353±0.788	10.25	98.3 %

Each value means the mean ± standard deviation, and the value in bold indicates the best performance

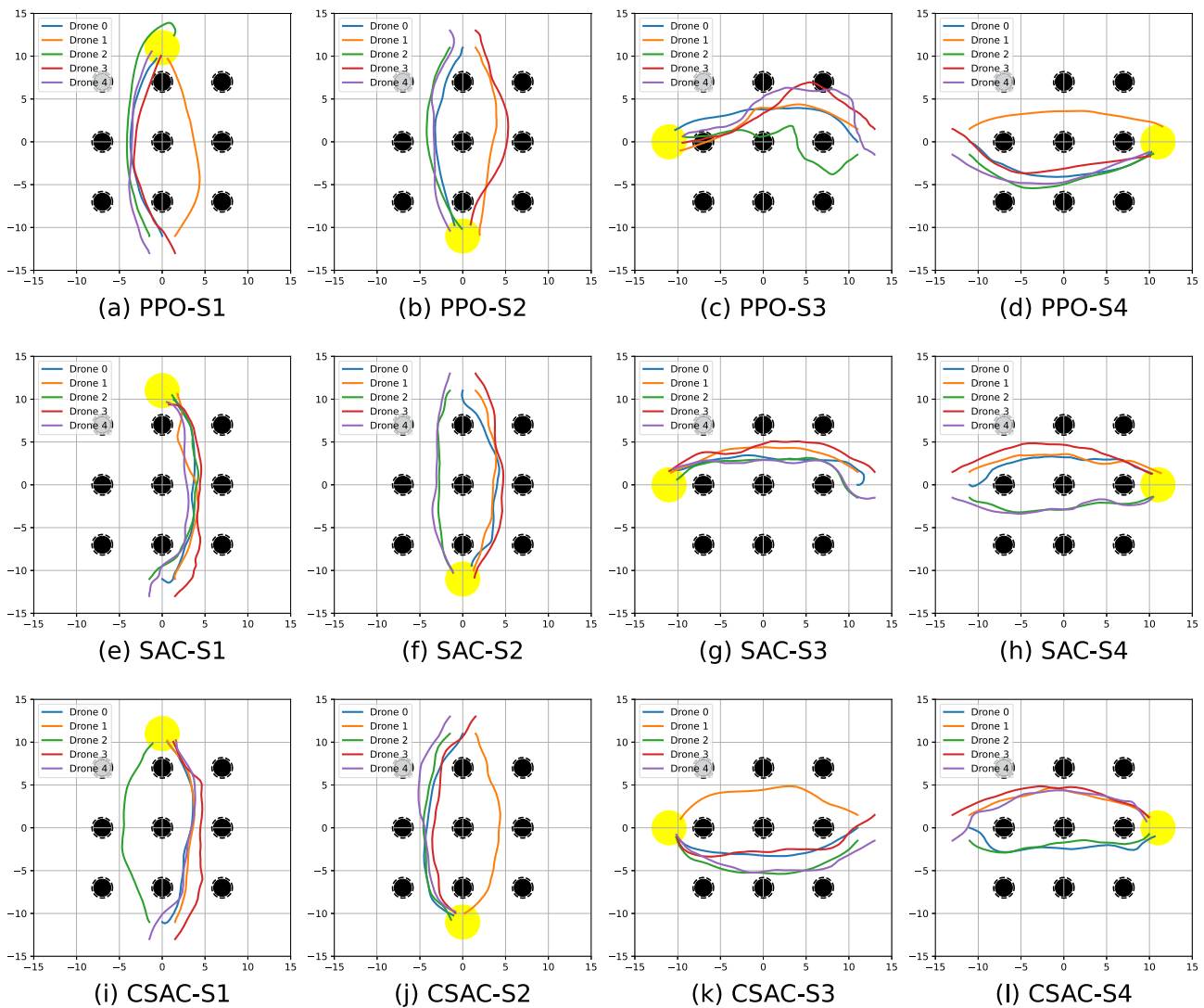


Fig. 6 Trajectories of the drone swarm with different policies

accomplish the rescue mission, with PPO exhibiting smoother trajectories.

In order to further compare and analyze the effectiveness of these algorithms, we visualize the speed of drone swarms in different scenarios, as shown in Fig. 7. From Fig. 7, compared horizontally, CSAC has the steadiest velocity change. PPO's constant acceleration towards the rescue area risks drone collisions, reducing the success rate. CSAC first accelerates UAVs to the maximum safe speed, maintaining it for a smooth approach and then slowing down near the target for safety and speed efficiency. SAC, despite a similar trend, has severe oscillations that hinder safe task execution, explaining its lower success rate than CSAC.

In the vertical comparison across different environments, CSAC shows the strongest robustness. PPO's varying acceleration trends, including deceleration

segments (Fig. 7g), correspond to collisions in Fig. 6c, resulting in a long mission time of 24.75 s. SAC and CSAC have similar trends, but the speed range (light blue shaded area) indicates SAC has significant speed differences among individuals, while CSAC performs better. This proves that CSAC considers both individual performance and group coordination when guiding the drone swarm to complete the mission.

From the perspective of model structure design, PPO adopts an Actor-Critic framework based on policy gradient theory, achieving policy iteration stability through policy clipping and an adaptive KL divergence penalty mechanism. However, the algorithm's policy update is entirely dependent on trajectory samples generated by the current policy, leading to relatively limited sample efficiency, which becomes the core limitation for its slower convergence speed. In contrast, SAC is based on the maximum

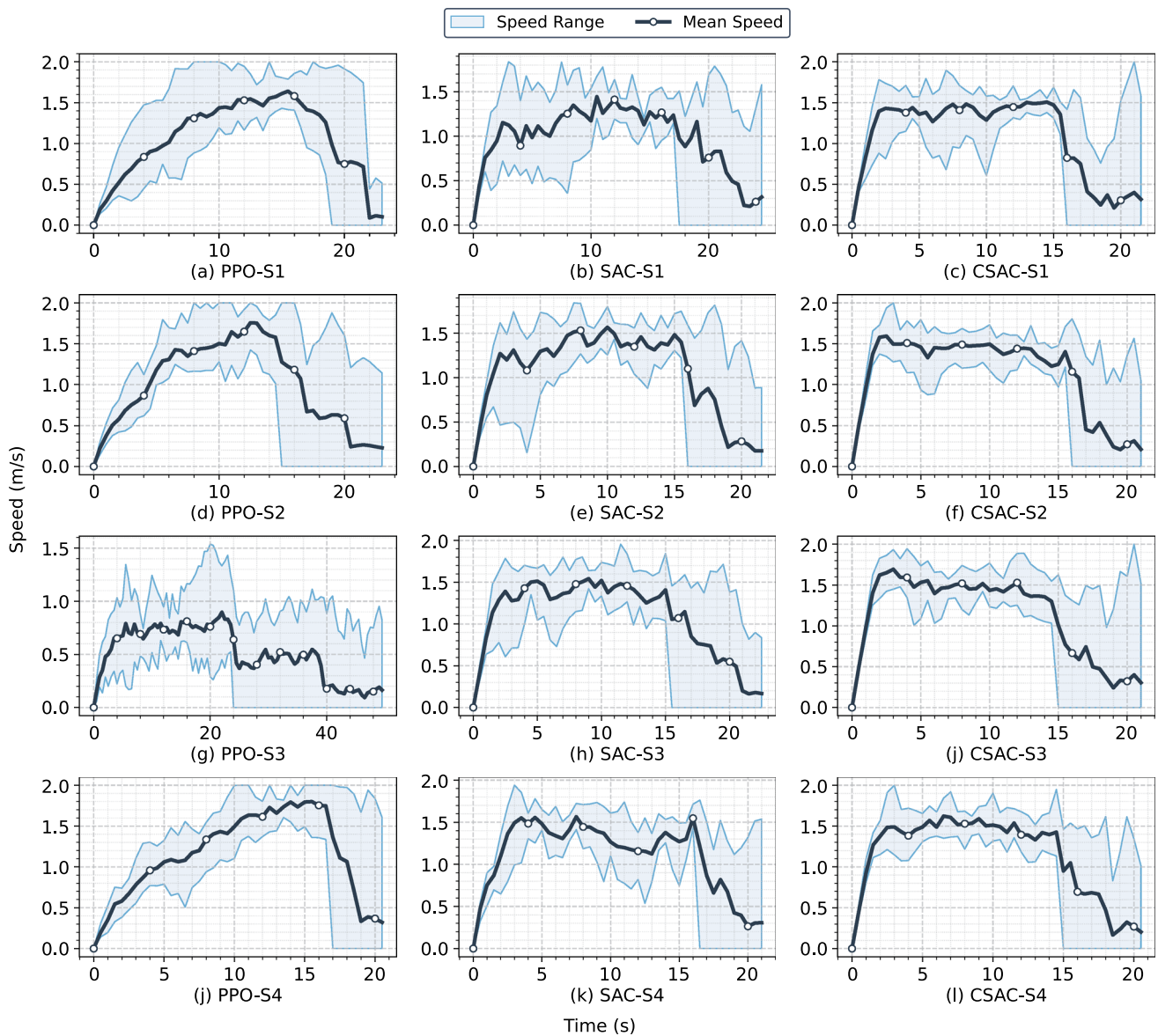


Fig. 7 Speeds of the drone swarm with different policies

entropy reinforcement learning framework, introducing a double Q-network architecture and random policy optimization mechanisms. Its off-policy characteristic supports the reuse of historical data, significantly improving sample utilization. Additionally, the entropy maximization objective function endows the algorithm with stronger exploration capabilities, making it superior in complex dynamic environments. However, the parallel structure design of multiple networks significantly increases computational overhead, constraining real-time decision-making efficiency. The proposed CSAC algorithm for this study, based on the SAC framework, optimizes the quantities of model parameters by integrating lightweight convolutional architecture techniques, reducing the number of model parameters by 92.91% and decreasing the ineffective

exploration time, ultimately achieving a significant improvement in convergence speed.

Overall, CSAC ensures the most compact formation and the smallest distance between drones and obstacles while completing the rescue mission at the fastest speed and guaranteeing the highest success rate. Building upon the strengths of SAC, CSAC retains the excellent performance of SAC and further accelerates the model's decision-making time, which is about 5.90%, enhancing the model's stability.

5 Conclusion

For the mission of drone swarms autonomous collaborative rescue, we formulate it as a POMDP problem. Considering the problems of excessive model parameters and long decision time, we propose CSAC. It inherits the excellent characteristics of SAC that can balance exploration and exploitation, while also reducing the number of parameters of the model. To validate our model, we set up the simulation environments by refining and designing the observation space, action space and reward function. Through simulation experiments to mimic the mission scenarios, we compare our model with baseline models. Compared to SAC, the results show that CSAC reduces the number of parameters by 92.91% while shortening the decision time by 5.90%. The results demonstrate that our model not only retains the good performance of SAC, but also significantly reduces the number of model parameters, with faster decision-making and more stable task execution capabilities.

For future research, firstly, We plan to extend the model to three-dimensional scenes. Secondly, we will further refine our simulation environment to bring it closer to the physical world, for example, addressing dynamic and heterogeneous obstacles to further bridge the gap between simulation and real-world deployment. In addition, we will further focus on the transition from the simulation environment to the physical world.

Author contributions Conceptualization, X. Z., Z.Z., J.T. and Y.W.; methodology, X.Z.; software, X.Z. and Z.Z.; validation, X.Z., Z.Z. and J.T.; formal analysis, X.Z., Y.W. and Z.Z.; investigation, X.Z., Z.Z., Y.W. and J.T.; resources, X.Z., Z.Z. and Y.W.; data curation, X.Z., Z.Z., Y.W. and J.T.; writing-original draft preparation, X.Z.; writing-review and editing, X.Z.; visualization, X.Z., Z.Z., Y.W. and J.T.; supervision, Y.W.; project administration, Y.W.; funding acquisition, J.T. All authors have read and agreed to the published version of the manuscript.

Funding This research was funded by the National Natural Science Foundation of China under Grants No.62073330.

Data availability The data are unavailable due to privacy or ethical restrictions.

Declarations

Conflict of interest The authors declare no conflicts of interest.

References

1. Tomic, T., Schmid, K., Lutz, P., Domel, A., Kassecker, M., Mair, E., Grix, I.L., Ruess, F., Suppa, M., Burschka, D.: Toward a fully autonomous uav: research platform for indoor and outdoor urban search and rescue. *IEEE Robot. Autom. Mag.* **19**(3), 46–56 (2012)
2. Baxter, J.L., Burke, E., Garibaldi, J.M., Norman, M.: Multi-robot search and rescue: a potential field based approach. *Auton. Robots Agents* (2007). https://doi.org/10.1007/978-3-540-73424-6_2
3. Tonasso, R., Tataru, D., Rauch, H., Pozsgay, V., Pfeiffer, T., Uythoven, E., Rodríguez-Martínez, D.: A lunar reconnaissance drone for cooperative exploration and high-resolution mapping of extreme locations. *Acta Astronaut.* **218**, 1–17 (2024)
4. Lee, M., Choi, M., Yang, T., Kim, J., Kim, J., Kwon, O., Cho, N.: A study on the advancement of intelligent military drones: focusing on reconnaissance operations. *IEEE Access* (2024). <https://doi.org/10.1109/ACCESS.2024.3390035>
5. Moon, S., Jeon, J., Kim, D., Kim, Y.: Swarm reconnaissance drone system for real-time object detection over a large area. *IEEE Access* **11**, 23505–23516 (2023)
6. Radmanesh, M., Sharma, B., Kumar, M., French, D.: Pde solution to uav/ugv trajectory planning problem by spatio-temporal estimation during wildfires. *Chin. J. Aeronaut.* **34**(5), 601–616 (2021)
7. Zhang, J., Wang, T., Wang, J., Du, W., Zheng, D., Wang, S., Li, Y.: Multi-uav collaborative surveillance network recovery via deep reinforcement learning. *IEEE Internet Things J.* (2024). <https://doi.org/10.1109/JIOT.2024.3446878>
8. Wang, Z., Hu, T., Long, L.: Multi-uav safe collaborative transportation based on adaptive control barrier function. *IEEE Trans. Syst. Man, Cybern. Syst.* **53**(11), 6975–6983 (2023)
9. Yu, Y., Lee, S.: Efficient multi-uav path planning for collaborative area search operations. *Appl. Sci.* **13**(15), 8728 (2023)
10. Chen, Z., Huang, Z., Zhang, J., Cheng, H., Li, J.: Resource allocation and collaborative offloading in multi-uav-assisted iov with federated deep reinforcement learning. *IEEE Internet Things J.* (2024). <https://doi.org/10.1109/JIOT.2024.3516838>
11. Bany Salameh, H., Hussienat, A., Alhafnawi, M., Al-Ajlouni, A.: Autonomous uav-based surveillance system for multi-target detection using reinforcement learning. *Clust. Comput.* **27**(7), 9381–9394 (2024)
12. Zhang, B., Jing, T., Lin, X., Cui, Y., Zhu, Y., Zhu, Z.: Deep reinforcement learning-based collaborative multi-uav coverage path planning. In: *Journal of Physics: Conference Series*. 2833, 012017 (2024). IOP Publishing
13. Wu, J., Luo, C., Luo, Y., Li, K.: Distributed uav swarm formation and collision avoidance strategies over fixed and switching topologies. *IEEE Trans. Cybern.* **52**(10), 10969–10979 (2021)
14. Tang, J., Lao, S., Wan, Y.: Systematic review of collision-avoidance approaches for unmanned aerial vehicles. *IEEE Syst. J.* **16**(3), 4356–4367 (2022). <https://doi.org/10.1109/JSYST.2021.3101283>
15. Loquercio, A., Maqueda, A.I., Del-Blanco, C.R., Scaramuzza, D.: Dronet: learning to fly by driving. *IEEE Robot. Autom. Lett.* **3**(2), 1088–1095 (2018)
16. Dai, X., Mao, Y., Huang, T., Qin, N., Huang, D., Li, Y.: Automatic obstacle avoidance of quadrotor uav via cnn-based learning. *Neurocomputing* **402**, 346–358 (2020)
17. Li, S., Öztürk, E., De Wager, C., De Croon, G.C., Izzo, D.: Aggressive online control of a quadrotor via deep network representations of optimality principles. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6282–6287 (2020). IEEE
18. Wang, D., Li, W., Liu, X., Li, N., Zhang, C.: Uav environmental perception and autonomous obstacle avoidance: a deep learning and depth camera combined solution. *Comput. Electron. Agric.* **175**, 105523 (2020)
19. Singla, A., Padakandla, S., Bhatnagar, S.: Memory-based deep reinforcement learning for obstacle avoidance in uav with limited

- environment knowledge. *IEEE Trans. Intell. Transp. Syst.* **22**(1), 107–118 (2019)
20. Fei, W., Xiaoping, Z., Zhou, Z., Yang, T.: Deep-reinforcement-learning-based uav autonomous navigation and collision avoidance in unknown environments. *Chin. J. Aeronaut.* **37**(3), 237–257 (2024)
 21. Ouahouah, S., Bagaa, M., Prados-Garzon, J., Taleb, T.: Deep-reinforcement-learning-based collision avoidance in uav environment. *IEEE Internet Things J.* **9**(6), 4015–4030 (2021)
 22. Xie, Z., Dames, P.: Drl-vo: learning to navigate through crowded dynamic scenes using velocity obstacles. *IEEE Trans. Robot.* **39**(4), 2700–2719 (2023)
 23. Sonny, A., Yeduri, S.R., Cenkeramaddi, L.R.: Q-learning-based unmanned aerial vehicle path planning with dynamic obstacle avoidance. *Appl. Soft Comput.* **147**, 110773 (2023)
 24. Wu, J., Wang, H., Liu, Y., Zhang, M., Wu, T.: Learning-based fixed-wing uav reactive maneuver control for obstacle avoidance. *Aerosp. Sci. Technol.* **126**, 107623 (2022)
 25. Azzam, R., Chehadeh, M., Hay, O.A., Humais, M.A., Boiko, I., Zweiri, Y.: Learning-based navigation and collision avoidance through reinforcement for uavs. *IEEE Trans. Aerosp. Electron. Syst.* (2023). <https://doi.org/10.1109/TAES.2023.3294889>
 26. Raja, G., Anbalagan, S., Narayanan, V.S., Jayaram, S., Ganapathisubramanian, A.: Inter-uav collision avoidance using deep-q-learning in flocking environment. In: 2019 IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), pp. 1089–1095 (2019). IEEE
 27. Wang, X., Gursoy, M.C., Erpek, T., Sagduyu, Y.E.: Learning-based uav path planning for data collection with integrated collision avoidance. *IEEE Internet Things J.* **9**(17), 16663–16676 (2022)
 28. Jang, K., Pant, Y.V., Rodionova, A., Mangharam, R.: Learning-to-fly rl: Reinforcement learning-based collision avoidance for scalable urban air mobility. In: 2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC), pp. 1–10 (2020). IEEE
 29. Hsu, Y.-H., Gau, R.-H.: Reinforcement learning-based collision avoidance and optimal trajectory planning in uav communication networks. *IEEE Trans. Mob. Comput.* **21**(1), 306–320 (2020)
 30. Zhao, Y., Guo, J., Bai, C., Zheng, H.: Reinforcement learning-based collision avoidance guidance algorithm for fixed-wing uavs. *Complexity* **2021**(1), 8818013 (2021)
 31. Schilling, F., Schiano, F., Floreano, D.: Vision-based drone flocking in outdoor environments. *IEEE Robot. Autom. Lett.* **6**(2), 2954–2961 (2021)
 32. Bai, C., Yan, P., Piao, H., Pan, W., Guo, J.: Learning-based multi-uav flocking control with limited visual field and instinctive repulsion. *IEEE Trans. Cybern.* **54**(1), 462–475 (2023)
 33. Wang, X., Gursoy, M.C.: Learning-based uav trajectory optimization with collision avoidance and connectivity constraints. *IEEE Trans. Wirel. Commun.* **21**(6), 4350–4363 (2021)
 34. Xu, L., Zhang, X., Xiao, D., Liu, B., Liu, A.: Research on heterogeneous multi-uav collaborative decision-making method based on improved ppo. *Appl. Intell.* **54**(20), 9892–9905 (2024)
 35. Yan, C., Wang, C., Xiang, X., Low, K.H., Wang, X., Xu, X., Shen, L.: Collision-avoiding flocking with multiple fixed-wing uavs in obstacle-cluttered environments: a task-specific curriculum-based madrl approach. *IEEE Trans. Neural Netw. Learn. Syst.* (2023). <https://doi.org/10.1109/TNNLS.2023.3245124>
 36. Fang, Z., Jiang, D., Huang, J., Cheng, C., Sha, Q., He, B., Li, G.: Autonomous underwater vehicle formation control and obstacle avoidance using multi-agent generative adversarial imitation learning. *Ocean Eng.* (2022). <https://doi.org/10.1016/j.oceaneng.2022.112182>
 37. Wan, Y., Tang, J., Zhao, Z.: Imitation learning of complex behaviors for multiple drones with limited vision. *Drones* **7**(12), 704 (2023)
 38. Wan, Y., Tang, J., Lao, S., Zhao, Z.: A distributed autonomous system for multi-uavs with limited visualization: employing dual-horizon nmpc controller. *IEEE Trans. Aerosp. Electron. Syst.* (2024). <https://doi.org/10.1109/TAES.2024.3411563>
 39. Lanier, M., Xu, Y., Jacobs, N., Zhang, C., Vorobeychik, Y.: Learning interpretable policies in hindsight-observable pomdps through partially supervised reinforcement learning. *arXiv preprint arXiv:2402.09290* (2024)
 40. Huang, H., Zhu, G., Fan, Z., Zhai, H., Cai, Y., Shi, Z., Dong, Z., Hao, Z.: Vision-based distributed multi-uav collision avoidance via deep reinforcement learning for navigation. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 13745–13752 (2022). IEEE

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Xiangyu Zhao is currently working as M.E. at the College of Systems Engineering, National University of Defense Technology, Changsha, China. His research focuses on the unmanned aerial vehicle system and path planning. He received his B.E. degree from the College of Artificial Intelligence.



Zipeng Zhao is currently working as Ph.D. at the College of Systems Engineering, National University of Defense Technology, Changsha, China. His research focuses on the unmanned aerial vehicle system and path planning. He received his B.E. degree from the College of Information Science and Engineering.



Yu Wan is currently working as a lecturer at the College of Systems Engineering, National University of Defense Technology, Changsha, China. His research focuses on the air traffic management and unmanned aerial vehicle system, such as collision avoidance of aircraft. He received his B.E. degree, M.E. degree and Ph.D. degree from the NUDT, in 2017, 2020 and 2024.



Jun Tang is currently an associate professor in the Science and Technology on Information Systems Engineering Laboratory, National University of Defense Technology. He is the winner of William Sweet Smith Prize in 2015. He was dedicated to the PhD research in the Technical Innovation Cluster on Aeronautical Management, Universitat Autònoma de Barcelona (UAB). His research interests focus on logistic systems, causal modelling, state

space, air traffic management and discrete event simulation (DES).

He has been very active in the simulation community, organizing as general co-chair for several international conferences. He acted as one of the main participants playing an important role in the FP7 European project “INnovative TEchnologies and Researches for a new Airport Concept towards Turnaround coordinatION (INTERACTION)” with collaboration of Airbus.